

# HARDWARE LABORATORY 1

## TECHNOLOGIES FOR IOT ECOSYSTEMS



**Politecnico  
di Torino**



**Prof. Luca Barbierato**

Assistant Professor

---

**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica

# EXERCISE 1

## TECHNOLOGIES FOR IOT ECOSYSTEMS



Politecnico  
di Torino

**Prof. Luca Barbierato**

Assistant Professor



**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica

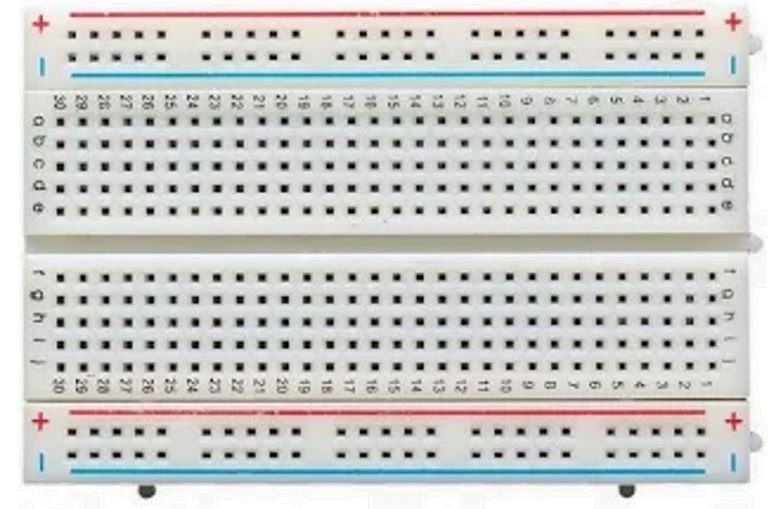
# Exercise 1

## HW components involved:

- Breadboard
- LEDs
- Resistors

## SW components involved:

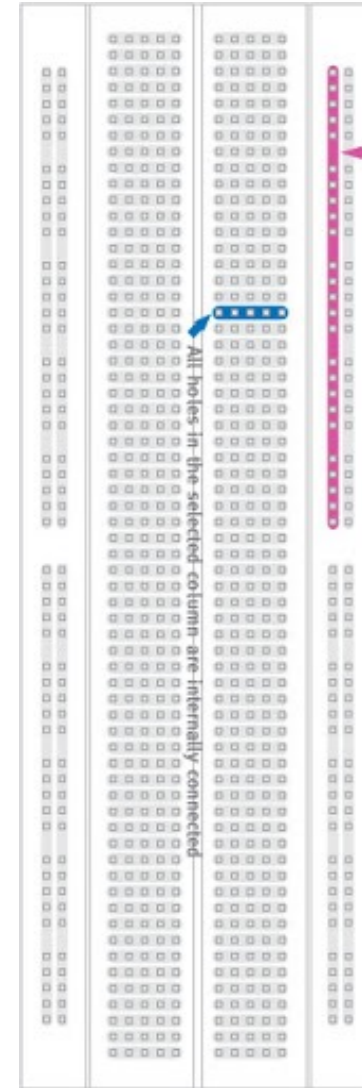
- Digital GPIO outputs
- Interrupts



# Breadboard

A breadboard allows you to build and test simple electronic circuits **without soldering**. Widely used for education, prototyping, and early-stage hardware development:

- Internal conductive strips connect rows of holes
- Ideal for rapid experimentation and debugging
- Power rails typically provided on the sides
- Suitable for low-voltage, low-current circuits
- Not recommended for high-frequency or high-power designs

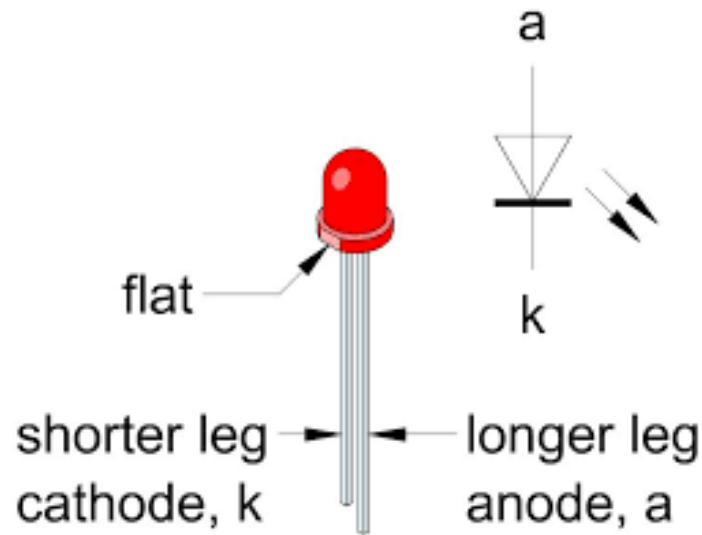


Normally  
for Vcc  
and GND

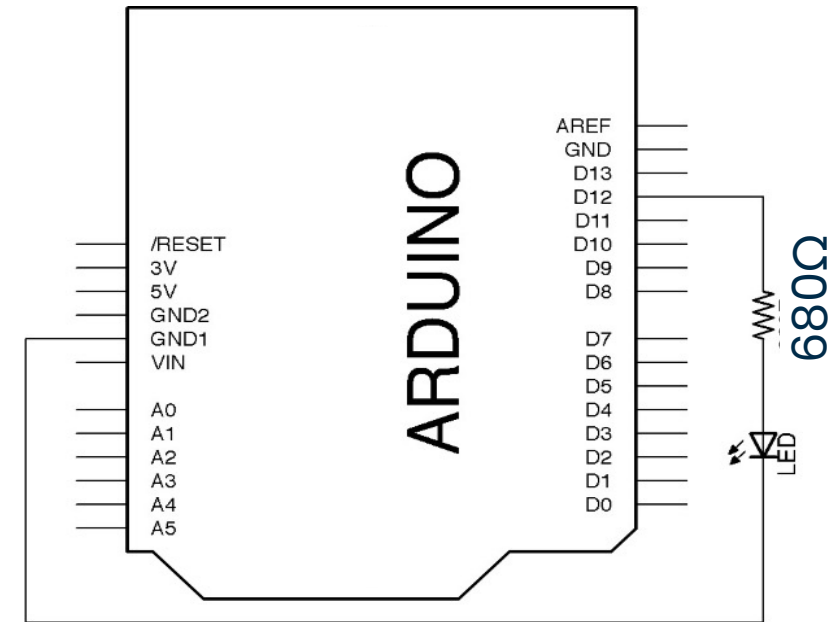
Normally  
for circuits

# Lead Connection

An LED is a diode, so a **series resistor** is required to limit the current flowing through it (i.e. to prevent excessive current that would effectively create a short path between VCC and GND). Without a current-limiting resistor, the LED (being a nonlinear device with low dynamic resistance when forward-biased) could draw excessive current and fail.



**Caution: longer leg is the anode**



# Resistor Color Coding

Our Arduino board provides a 3.3 V GPIO HIGH level.

Using a **680  $\Omega$**  series resistor and assuming an LED forward voltage of **0.6 V**, the current is:

$$I = \frac{3.3V - 0.6V}{680 \Omega} = \frac{2.7V}{680 \Omega} \approx 3.97 \text{ mA}$$

$$I \approx 3.9 \text{ mA} < 8 \text{ mA}$$

This value is safely below the maximum allowable GPIO current ( $\approx 8 \text{ mA}$  per pin), ensuring reliable and safe operation.



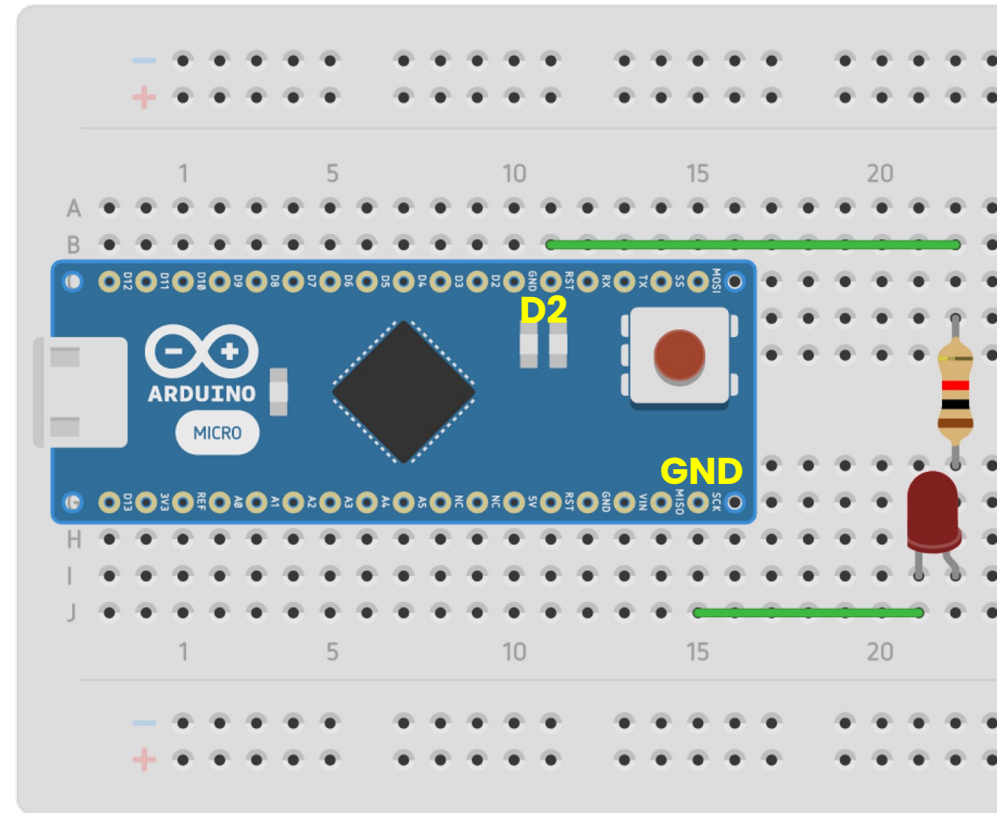
| COLOR  | 1 <sup>ST</sup> BAND | 2 <sup>ND</sup> BAND | 3 <sup>RD</sup> BAND | MULTIPLIER    | TOLERANCE        |
|--------|----------------------|----------------------|----------------------|---------------|------------------|
| Black  | 0                    | 0                    | 0                    | 1 $\Omega$    |                  |
| Brown  | 1                    | 1                    | 1                    | 10 $\Omega$   | $\pm 1\%$ (F)    |
| Red    | 2                    | 2                    | 2                    | 100 $\Omega$  | $\pm 2\%$ (G)    |
| Orange | 3                    | 3                    | 3                    | 1K $\Omega$   |                  |
| Yellow | 4                    | 4                    | 4                    | 10K $\Omega$  |                  |
| Green  | 5                    | 5                    | 5                    | 100K $\Omega$ | $\pm 0.5\%$ (D)  |
| Blue   | 6                    | 6                    | 6                    | 1M $\Omega$   | $\pm 0.25\%$ (C) |
| Violet | 7                    | 7                    | 7                    | 10M $\Omega$  | $\pm 0.10\%$ (B) |
| Grey   | 8                    | 8                    | 8                    | 100M $\Omega$ | $\pm 0.05\%$     |
| White  | 9                    | 9                    | 9                    | 1G $\Omega$   |                  |
| Gold   |                      |                      |                      | 0.1 $\Omega$  | $\pm 5\%$ (J)    |
| Silver |                      |                      |                      | 0.01 $\Omega$ | $\pm 10\%$ (K)   |



**Caution: some of you might have a 5-band resistor, but the concept is similar!**

# LED Connection

Example of breadboard connection:



# Code: Version 1

MBED\_Rpi\_Pico\_TimerInterrupt.h library allows attaching interrupt service routines to the board's Timers.

- See documentation or check internal libraries!

GPIO pins are identified by integer constants:

- D12 -> 12

Global constants for the switching semi-period (in ms):

- L suffix for "long int" constants
- Probably superfluous on 32-bit MCU

Global variables to store the current state of the LEDs:

- LOW and HIGH MACROS for 0 and 1, respectively

Create an instance of the timer class using Timer1:

- Timers 0-3 are available
- Careful, some timers are used for other functions; check the documentation.

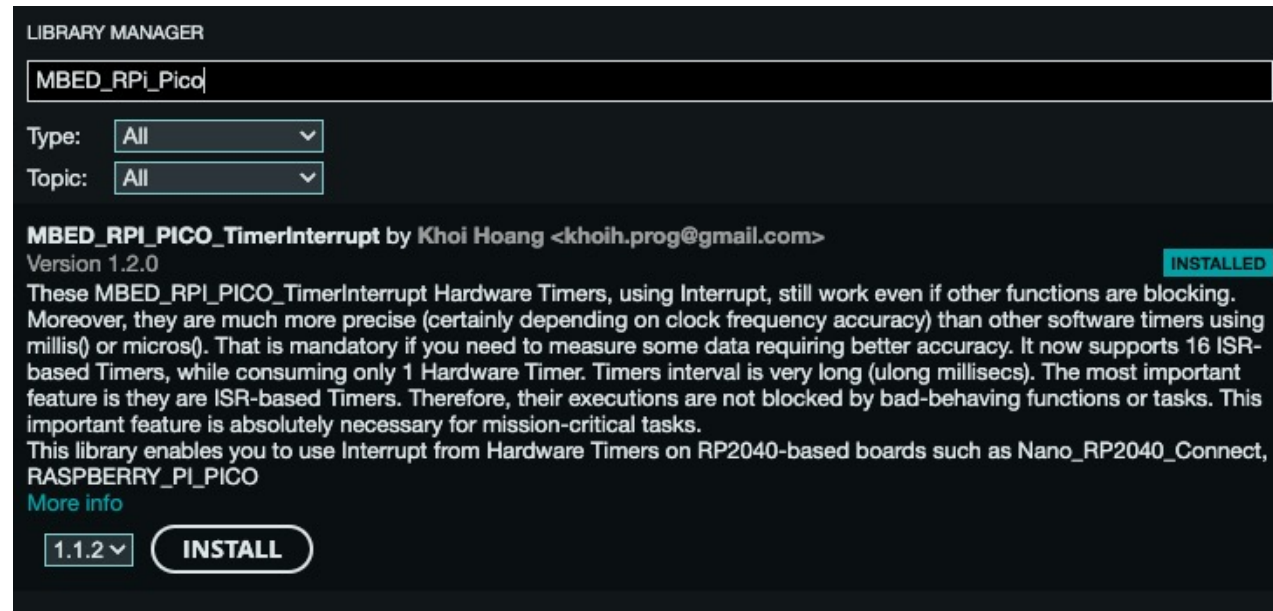
```
1  #include <MBED_RPi_Pico_TimerInterrupt.h>
2
3  const int RLED_PIN = 2;
4  const int GLED_PIN = 3;
5
6  const long R_HALF_PERIOD = 1500L;
7  const long G_HALF_PERIOD = 3500L;
8
9  int redLedState = LOW;
10 int greenLedState = LOW;
11
12 MBED_RPI_PICO_Timer ITimer1(1);
```



# Install libraries

The Timer library used in the previous code is probably not installed by default. Install procedure:

- Go to Tools/Manage Libraries
- Filter by name
- Click Install



# Code: Version 1

In the `setup()` function:

- set the two LED pins as outputs
- Attach an ISR function to the timer expiration

Careful, the timer library requires an interval in **microseconds**!

```
21 void setup() {  
22     pinMode(RLED_PIN, OUTPUT);  
23     pinMode(GLED_PIN, OUTPUT);  
24     ITimer1.setInterval(G_HALF_PERIOD * 1000, blinkGreen);  
25 }
```

# Code: Version 1

In the ISR:

- toggle one LED state asynchronously with respect to `loop()`

The ISR must be a void function that takes a `uint` parameter representing the timer number.

The library docs clarify that you must call `TIMER_ISR_START/TIMER_ISR_END` upon entry/exit.

```
14 void blinkGreen(uint alarm_num) {  
15     TIMER_ISR_START(alarm_num);  
16     digitalWrite(GLED_PIN, greenLedState);  
17     greenLedState = !greenLedState;  
18     TIMER_ISR_END(alarm_num);  
19 }  
20
```

# Code: Version 1

In the `loop()` function:

- toggle the other LED state using the `delay()` function

Careful: `delay()` wants a milliseconds value.

```
27 void loop() {  
28     digitalWrite(RLED_PIN, redLedState);  
29     redLedState = !redLedState;  
30     delay(R_HALF_PERIOD);  
31 }
```

# Complete Code: Version 1

As for all Software, there are many ways to obtain the same result.

For this first example, we will see some alternatives, so that you can learn about additional functionalities & libraries available for the board.

Might be useful for Part 2!

```
1  #include <MBED_RPi_Pico_TimerInterrupt.h>
2
3  const int RLED_PIN = 2;
4  const int GLED_PIN = 3;
5
6  const long R_HALF_PERIOD = 1500L;
7  const long G_HALF_PERIOD = 3500L;
8
9  int redLedState = LOW;
10 int greenLedState = LOW;
11
12 MBED_RPI_PICO_Timer ITimer1(1);
13
14 void blinkGreen(uint alarm_num) {
15     TIMER_ISR_START(alarm_num);
16     digitalWrite(GLED_PIN, greenLedState);
17     greenLedState = !greenLedState;
18     TIMER_ISR_END(alarm_num);
19 }
20
21 void setup() {
22     pinMode(RLED_PIN, OUTPUT);
23     pinMode(GLED_PIN, OUTPUT);
24     ITimer1.setInterval(G_HALF_PERIOD * 1000, blinkGreen);
25 }
26
27 void loop() {
28     digitalWrite(RLED_PIN, redLedState);
29     redLedState = !redLedState;
30     delay(R_HALF_PERIOD);
31 }
```

# Code: Version 2

Instead of the Timer Library, you can use the Scheduler library

- A simple multi-task scheduler for the Arduino

The scheduler is **non-preemptive!** Each task must release control to allow the others to run:

- Implicitly with a wait statement (e.g. `delay()`)
- Explicitly with the `yield()` function

See documentation!

# Code: Version 2

Include the `Scheduler.h` library:

```
1  #include <Scheduler.h>
2
```

Define constants and global variables as before.

In `setup()`, use the scheduler to start a secondary loop (`loop2`):

```
12  void setup() {
13      pinMode(RLED_PIN, OUTPUT);
14      pinMode(GLED_PIN, OUTPUT);
15      Scheduler.startLoop(loop2);
16  }
```

# Code: Version 2

Define the two loops, each performing a **short operation** and then releasing control through a `delay()` call.

```
18 void loop() {  
19     digitalWrite(RLED_PIN, redLedState);  
20     redLedState = !redLedState;  
21     delay(R_HALF_PERIOD);  
22 }  
23  
24 void loop2() {  
25     digitalWrite(GLED_PIN, greenLedState);  
26     greenLedState = !greenLedState;  
27     delay(G_HALF_PERIOD);  
28 }
```

**Default loop, always executed in all sketches**

**Additional loop, executed because we called `Scheduler.startLoop()`**

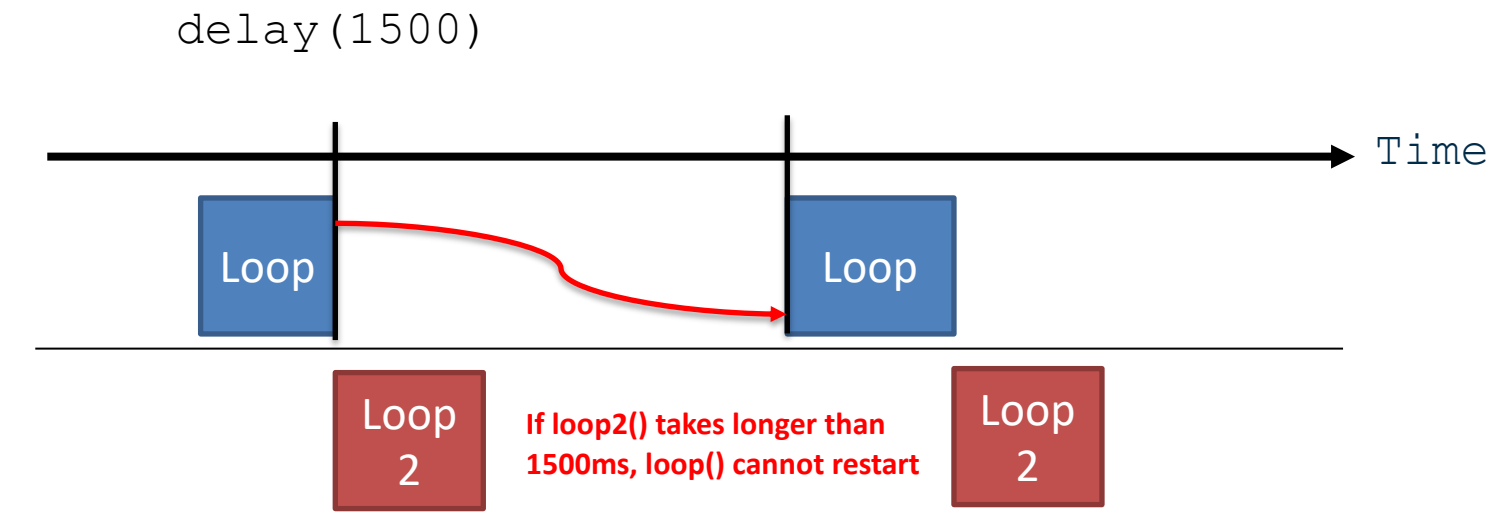


# Scheduler

The Arduino MCU is **single-core**. The loops are **not** running in parallel. They are using **time-sharing**.

This means that loops should relinquish control frequently to create the illusion of parallelism.

Different from the Timer solution (i.e. Version 1), no interruption.



# Complete Code: Version 2

The RP2040 also has an internal RGB LED. This exercise required you to use external LEDs so that you could also learn how to physically connect them to the board.

In future labs, you may also use the internal LED, e.g. as a status indicator or for debugging.

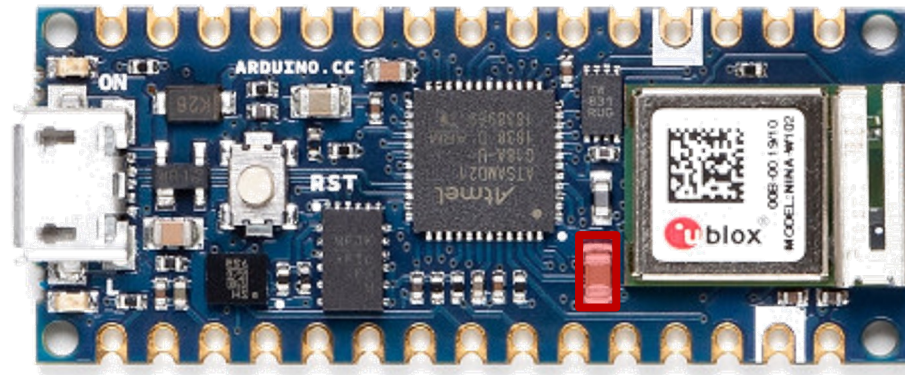
```
1  #include <Scheduler.h>
2
3  const int GLED_PIN = 2;
4  const int RLED_PIN = 3;
5
6  const long R_HALF_PERIOD = 1500L;
7  const long G_HALF_PERIOD = 3500L;
8
9  int redLedState = LOW;
10 int greenLedState = LOW;
11
12 void setup() {
13     pinMode(RLED_PIN, OUTPUT);
14     pinMode(GLED_PIN, OUTPUT);
15     Scheduler.startLoop(loop2);
16 }
17
18 void loop() {
19     digitalWrite(RLED_PIN, redLedState);
20     redLedState = !redLedState;
21     delay(R_HALF_PERIOD);
22 }
23
24 void loop2() {
25     digitalWrite(GLED_PIN, greenLedState);
26     greenLedState = !greenLedState;
27     delay(G_HALF_PERIOD);
28 }
```

# Code: Version 3

The RP2040 also has an internal RGB LED.

This exercise required you to use external LEDs so that you could also learn how to physically connect them to the board.

In future labs, you may also use the internal LED, e.g., as a status indicator or for debugging.



# Code: Version 3

There are some important differences between a normal GPIO (external LED) and the internal RGB LED in terms of programming.

First, the RGB led is connected to pins 16 and 18 on the WiFi Ublox NINA Module. To use the RGB LED, we need to go through the WiFi module.

In practice, this means installing (if needed) and including the WiFiNINA library:

```
3  #include <WiFiNINA.h>
```

## Code: Version 3

The three LED colors have a common anode pin, and are controlled from the cathode (i.e. active-low logic).

The three cathode pins are assigned to constants `LEDR`, `LEDG`, `LEDB` in your sketch.

If you want to rename the pins, you should use a define statement **not a const variable**, different w.r.t. GPIOs.

```
4  #define RLED_PIN LEDR
```

# Code: Version 3

For internal reasons, the `digitalWrite()` function for NINA pins is also slightly different w.r.t. the one for GPIOs.

Rather than an **int**, the written value should be a variable of type `PinStatus`, which is an enum type defined as follows:

```
typedef enum {  
    LOW      = 0,  
    HIGH     = 1,  
    CHANGE   = 2,  
    FALLING  = 3,  
    RISING   = 4,  
} PinStatus;
```

**Luckily, LOW and HIGH have the same values as for GPIOs**

# Code: Version 3

In practice, you can still write 0/1 to the pin with appropriate type-casting:

```
digitalWrite(RLED_PIN, (PinStatus) redLedState);
```

Lastly, and very importantly:

- The internal RGB LED cannot be accessed from within Interrupt Service Routines
- This will make your sketch crash

The rest of the code is identical to the case of GPIOs.

Extra: Try to solve Exercise 1 to use the internal LED.

# EXERCISE 2

## TECHNOLOGIES FOR IOT ECOSYSTEMS



Politecnico  
di Torino

**Prof. Luca Barbierato**

Assistant Professor



**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica



# Exercise 2

## **HW components involved:**

- Same as 1, plus Serial port

## **SW component involved:**

- Arduino Serial library

# Exercise 2

Create a Serial connection with a specified baud rate.

Wait until the connection is established, so the program will not start until you open the Serial monitor in the IDE

Print a newline-terminated string.

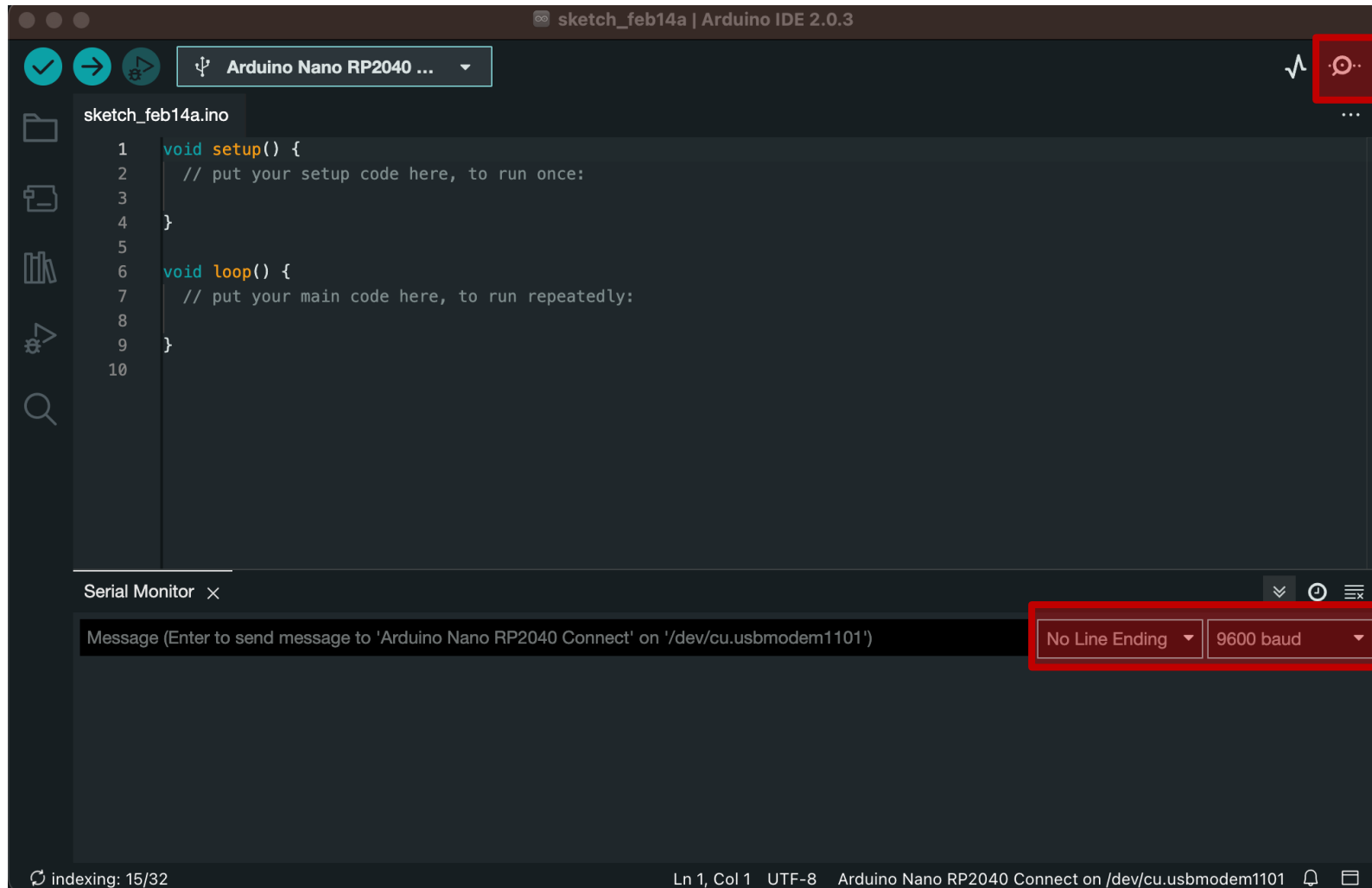
Check if bytes are available in the Serial buffer.

Read a single byte from the buffer.

```
void setup() {  
  Serial.begin(9600);  
  while (!Serial);  
  Serial.println("Lab 1.2 Starting");  
  //etc...|
```

```
void serialPrintStatus() {  
  if (Serial.available() > 0) {  
    int inByte = Serial.read();  
  
    if(inByte == ...) {  
      //etc|
```

# Serial Monitor



**Click here (or Tools/Serial Monitor) to open the monitor for the selected Port**

**Set the baud-rate to the same value used when creating the Serial() instance and select the termination character here**

# New Code Element

The full reference for the `Serial` class can be found [here](#):

- Many more utility functions to read/write various kinds of data
- Example: attach ISR when new data is available: `serialEvent()`
- Parse integers or floats from the buffer: `parseInt()`, `parseFloat()`
- Flush the buffer: `flush()`

Arduino can use char-based C-style strings, but also supports a more flexible [String](#) class:

Easier concatenation and processing... but **more memory!!**

# New Code Element

Important: variables used both by the `loop()` function and by an ISR should be declared as `volatile`:

- This qualifier tells the compiler to avoid register-based optimisations for the variable
- The variable is always written back to memory
- It prevents Read-After-Write dependency violations.

Additionally, if there are pieces of code that cannot be interrupted, you can use the `noInterrupts()` and `interrupts()` functions to disable and re-enable interrupts as needed.

# Exercise 2: Output Example



The screenshot shows a terminal window with a title bar containing three colored buttons (red, yellow, green) and the text `/dev/cu.usbmodem14201`. Below the title bar is a text input field and a `Send` button. The main area of the window displays the following text:

```
Lab 1.2 Starting  
LED 2 Status: 1  
LED 3 Status: 0  
LED 3 Status: 1  
Invalid command  
LED 2 Status: 1
```

At the bottom of the window is a control bar with the following elements from left to right:

- A checked checkbox labeled `Autoscroll`.
- An unchecked checkbox labeled `Show timestamp`.
- A dropdown menu showing `No line ending`.
- A dropdown menu showing `9600 baud`.
- A `Clear output` button.

# EXERCISE 3

## TECHNOLOGIES FOR IOT ECOSYSTEMS



**Politecnico  
di Torino**



**Prof. Luca Barbierato**

Assistant Professor

---

**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica

# Exercise 3

## HW components involved:

- PIR Sensor
- Serial port

## SW component involved:

- GPIO interrupts





# PIR Motion Sensor

You might have received two different types of PIR sensors in your kit. Both have similar pinouts and functionality. See the online datasheet.

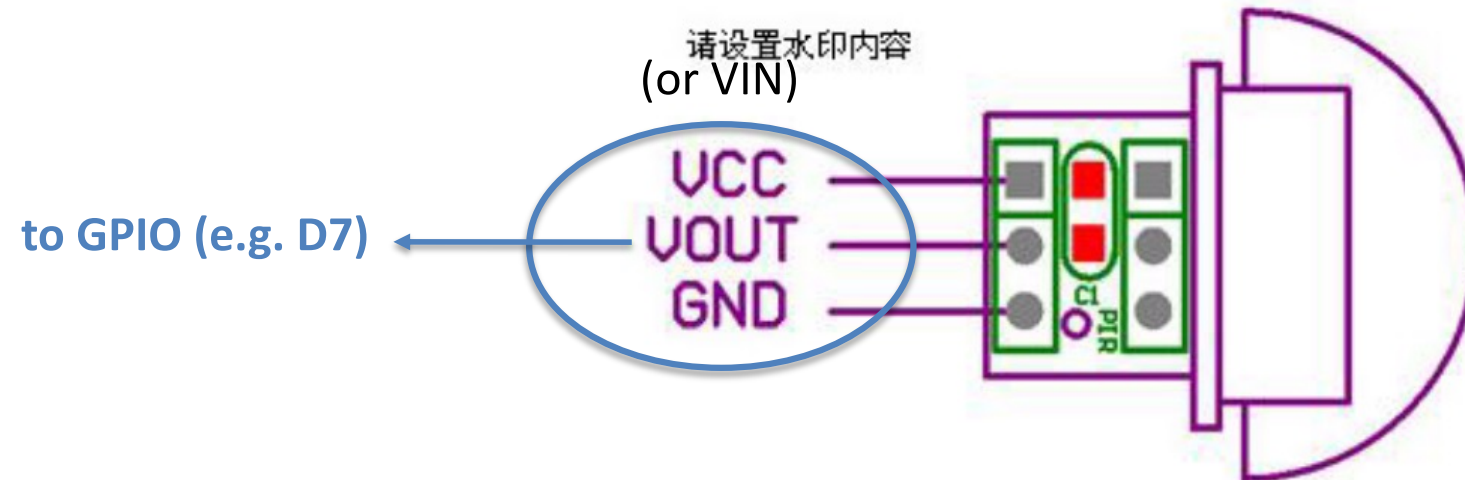


HC-SR312



HC-SR501

# HC-SR312

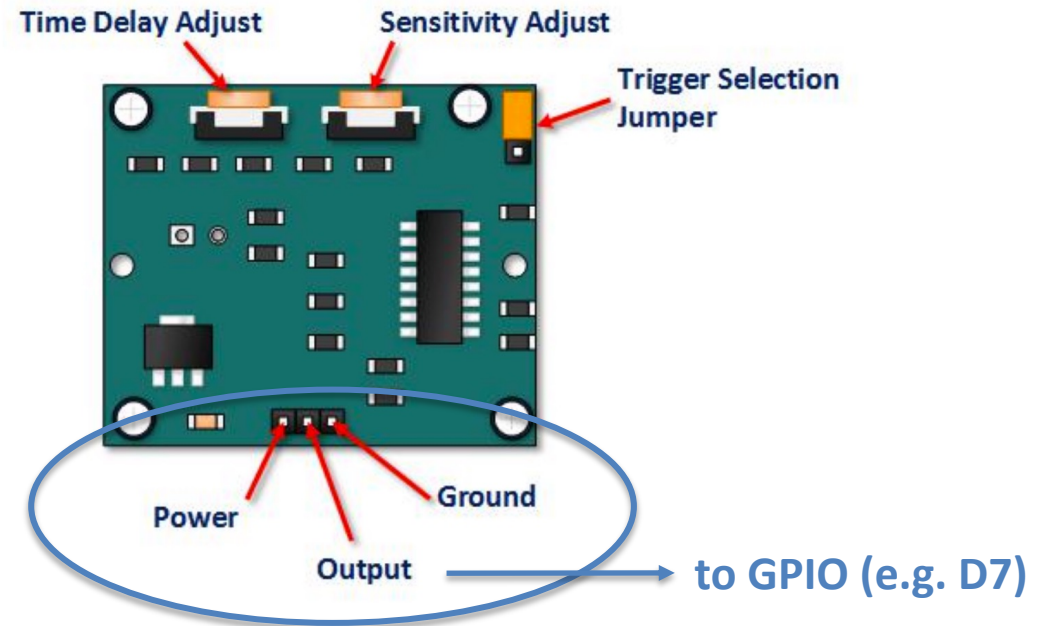


# HC-SR501

Two potentiometers to set the length of the detection and the sensitivity.

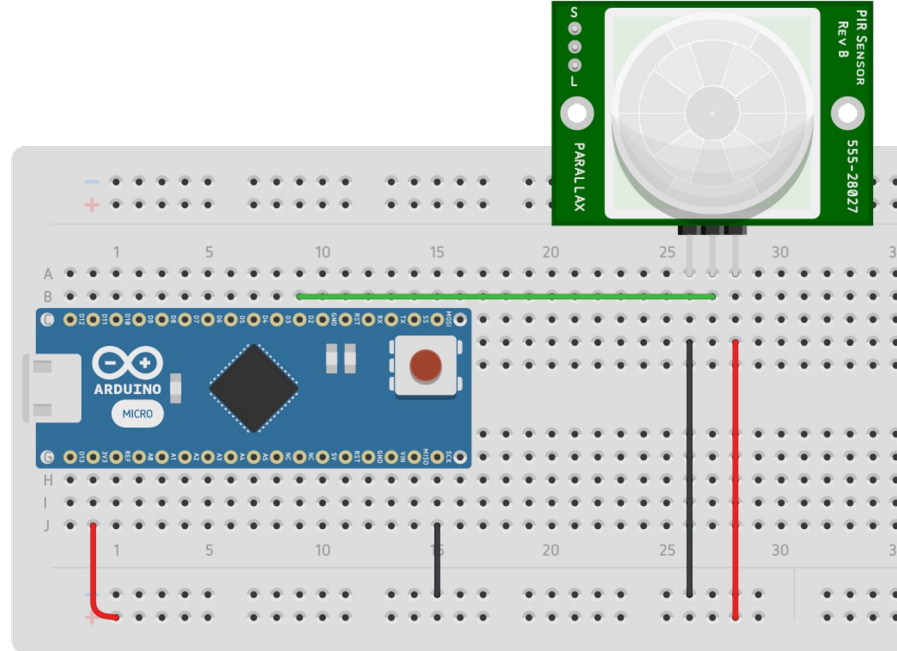
One jumper to select between 2 operating modes (single and repeatable trigger).

Use the datasheet to understand each configuration parameter.



# PIR Connection

Suggestion: add new components without disconnecting the previous ones (LEDs). In part 2, you'll use them all together.



**WARNING: this diagram is for our sensor. The pinout of the PIR on Tinkercad is different!**

# New Code Elements

All GPIO pins (except A6 and A7) support interrupts.

`tot_count` is touched by ISR and `loop()` → `volatile`

```
1  const int LED_PIN = 2;
2  const int PIR_PIN = 4;
3
4  volatile int tot_count = 0;
```

Attach the ISR function to the appropriate GPIO interrupt.

```
14  pinMode(PIR_PIN, INPUT);
15  attachInterrupt(digitalPinToInterrupt(PIR_PIN), checkPresence, CHANGE);
16  //etc...
```

**PIN number to Interrupt number**

**ISR function pointer**

**RISING, FALLING, CHANGE, etc.**

# Exercise 3: Output Example



```
/dev/cu.usbmodem14201  
Lab 1.3 starting  
Total people count: 0  
Total people count: 1  
Total people count: 1  
Total people count: 3
```

Autoscroll ☐ Show timestamp No line ending 9600 baud Clear output

# EXERCISE 4

## TECHNOLOGIES FOR IOT ECOSYSTEMS



Politecnico  
di Torino

**Prof. Luca Barbierato**

Assistant Professor



**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica

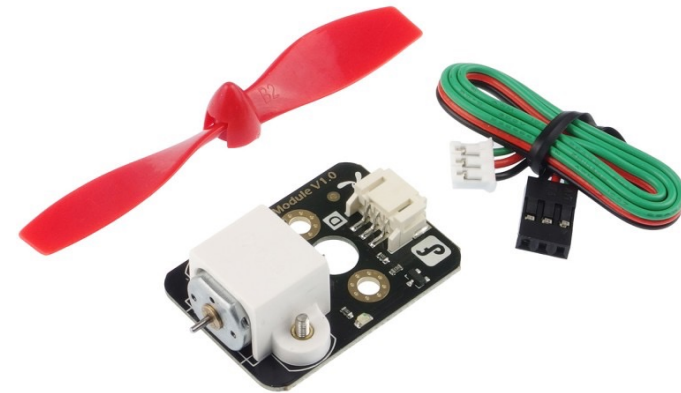
# Exercise 4

## HW components involved:

- DC Motor (fan)
- Serial port

## SW component involved:

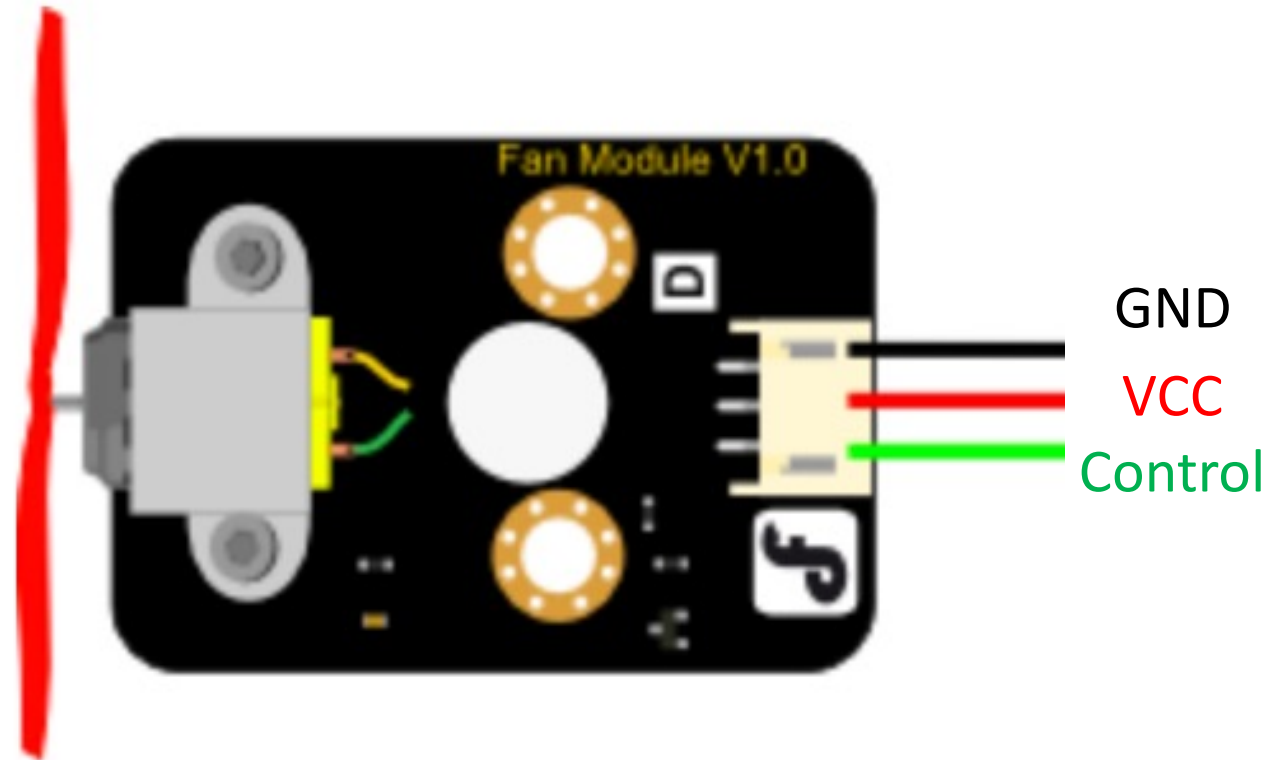
- PWM outputs





# PWM Fan

Breadboard connection very similar to PIR.



# New Code Element

All GPIO pins except A6 and A7 support PWM.

To specify a duty cycle, simply do an `analogWrite()` to the PIN.

DC values are integers between 0 (0%) and 255 (100%).

```
float current_speed = 0;
```

```
void setup() {  
    pinMode(FAN_PIN, OUTPUT);  
    analogWrite(FAN_PIN, (int) current_speed);  
    //etc.
```

# Exercise 4: Output Example



A screenshot of a terminal window titled `/dev/cu.usbmodem14201`. The window contains a list of speed control commands and their corresponding values. The output shows a sequence of increasing and decreasing speeds, starting from 204.00 and ending at 0.00. The terminal also features a 'Send' button at the top right and a status bar at the bottom with options for 'Autoscroll', 'Show timestamp', 'No line ending', '9600 baud', and a 'Clear output' button.

```
/dev/cu.usbmodem14201  
Increasing speed: 204.00  
Increasing speed: 229.50  
Increasing speed: 255.00  
Decreasing speed: 229.50  
Decreasing speed: 204.00  
Decreasing speed: 178.50  
Decreasing speed: 153.00  
Decreasing speed: 127.50  
Decreasing speed: 102.00  
Decreasing speed: 76.50  
Decreasing speed: 51.00  
Decreasing speed: 25.50  
Decreasing speed: 0.00  
Already at min speed  
Already at min speed
```

☒ Autoscroll ☐ Show timestamp    No line ending    9600 baud    Clear output

# EXERCISE 5

## TECHNOLOGIES FOR IOT ECOSYSTEMS



**Politecnico  
di Torino**



**Prof. Luca Barbierato**

Assistant Professor

---

**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica

# Exercise 5

## HW components involved:

- Temperature Sensor
- Serial port

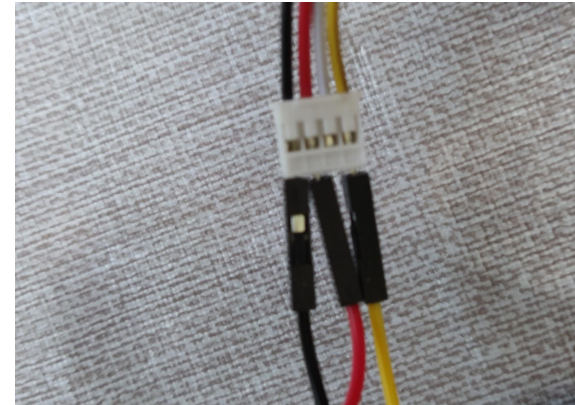
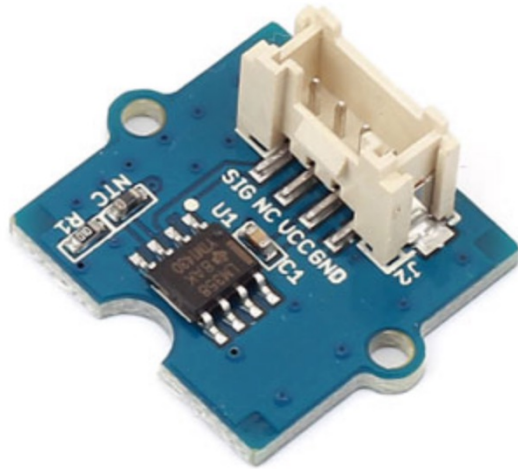
## SW component involved:

- Analog inputs



# Grove Temperature Sensor

Breadboard connection is again very similar to the previous sensors.  
You can plug your jumper cables into the output of the provided cable.



# Grove Temperature Sensor

The sensor produces an output **voltage** that depends on the temperature through a variable resistance (i.e. the thermistor).

We read the voltage, and we need to recover the temperature.

Two-step process:

1. From voltage to resistance
2. From resistance to temperature

The sensor has two main parameters:

- **$R_0 = 100\text{K}\Omega$** ; thermistor resistance at nominal  $T_0 = 25^\circ\text{C}$
- **$B = 4275\text{K}$** ; defines the shape of the (non-linear) relation between resistance and temperature

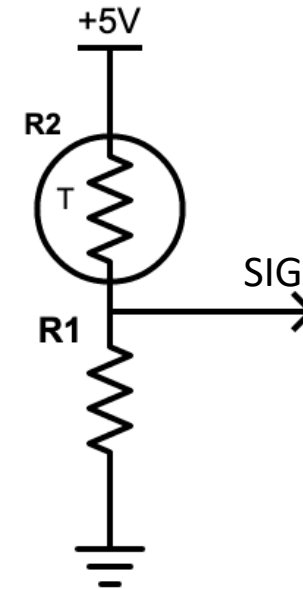
# Grove Temperature Sensor

Step 1:

$R1 = R0 = 100K\Omega$

We can extract  $R$  from  $V_{sig}$  using a simple resistive divider equation:

$$V_{sig} = \frac{R_1}{R + R_1} * VCC \quad \Rightarrow \quad R = \left( \frac{VCC}{V_{sig}} - 1 \right) * R_1$$



The Arduino ADCs have 10-bit resolution:

$GND = 0$ ,  $VCC = 1023$



# Grove Temperature Sensor

## Step 2:

The conversion from R to T can be obtained by inverting the definition of B from the datasheet:

$$B = \frac{\ln\left(\frac{R}{R_0}\right)}{\frac{1}{T} - \frac{1}{T_0}} \quad \Rightarrow \quad T = \frac{1}{\frac{\ln\left(\frac{R}{R_0}\right)}{B} + \left(\frac{1}{T_0}\right)}$$

**Warning:** In this equation, both T and T<sub>0</sub> are in Kelvin.

We want to print all temperature values in Celsius.

# New Code Element

Analog pins are identified with the MACRO Ax where x is the PIN number on the board.

```
const int TEMP_PIN = A1;  
const int B = 4275;  
const long int R0 = 100000;
```

Simply call `analogRead()` to read the voltage value from the sensor.

```
void loop() {  
    int a = analogRead(TEMP_PIN);  
}
```

# Exercise 5: Output Example

Touch the thermistor or blow on it to see the temperature change.



The screenshot shows a serial terminal window titled "/dev/cu.usbmodem14201". It features a text input field at the top with a "Send" button. The main area displays a series of temperature readings. The bottom of the window contains configuration options: "Autoscroll" (checked), "Show timestamp" (unchecked), "No line ending" (selected from a dropdown), "9600 baud" (selected from a dropdown), and a "Clear output" button.

```
Lab 1.5 starting
temperature = 18.38
temperature = 18.38
temperature = 19.74
temperature = 24.39
temperature = 22.78
temperature = 22.06
temperature = 21.50
temperature = 21.18
temperature = 20.86
temperature = 20.62
temperature = 20.38
```

☒ Autoscroll   ☐ Show timestamp   No line ending   9600 baud   Clear output

# EXERCISE 6

## TECHNOLOGIES FOR IOT ECOSYSTEMS



Politecnico  
di Torino

**Prof. Luca Barbierato**

Assistant Professor



**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica

# Exercise 6

## HW components involved:

- Temperature Sensor
- LCD Display (with I2C connection)

## SW component involved:

- LCD library (based on Wire)



# DF Robot I2C Display

Find the datasheet.

**Careful:** we won't use the library provided in the datasheet, but a standard library from the Arduino IDE's Library Manager, compatible with this display.

So don't copy the code! It won't work!

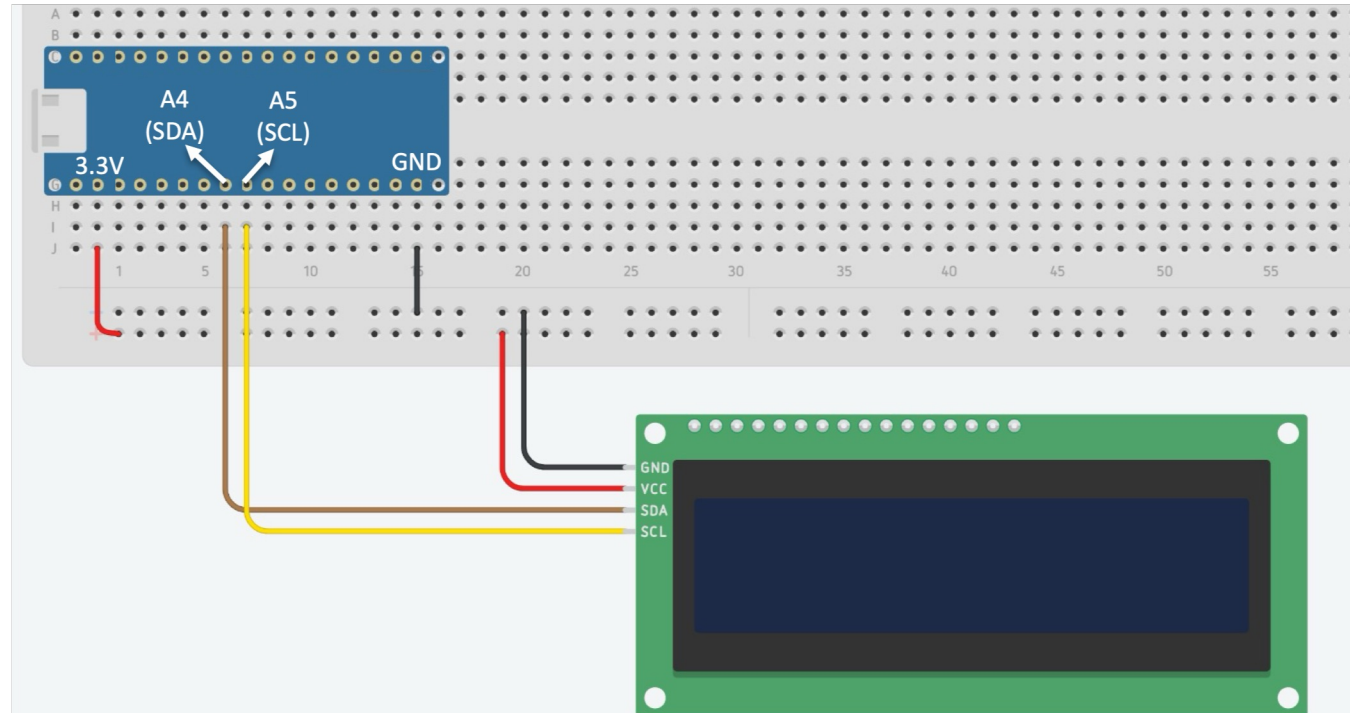


# DF Robot I2C Display Connection

The Arduino uses Pins A4 and A5 as SDA and SCL

Check pinout slide at the beginning

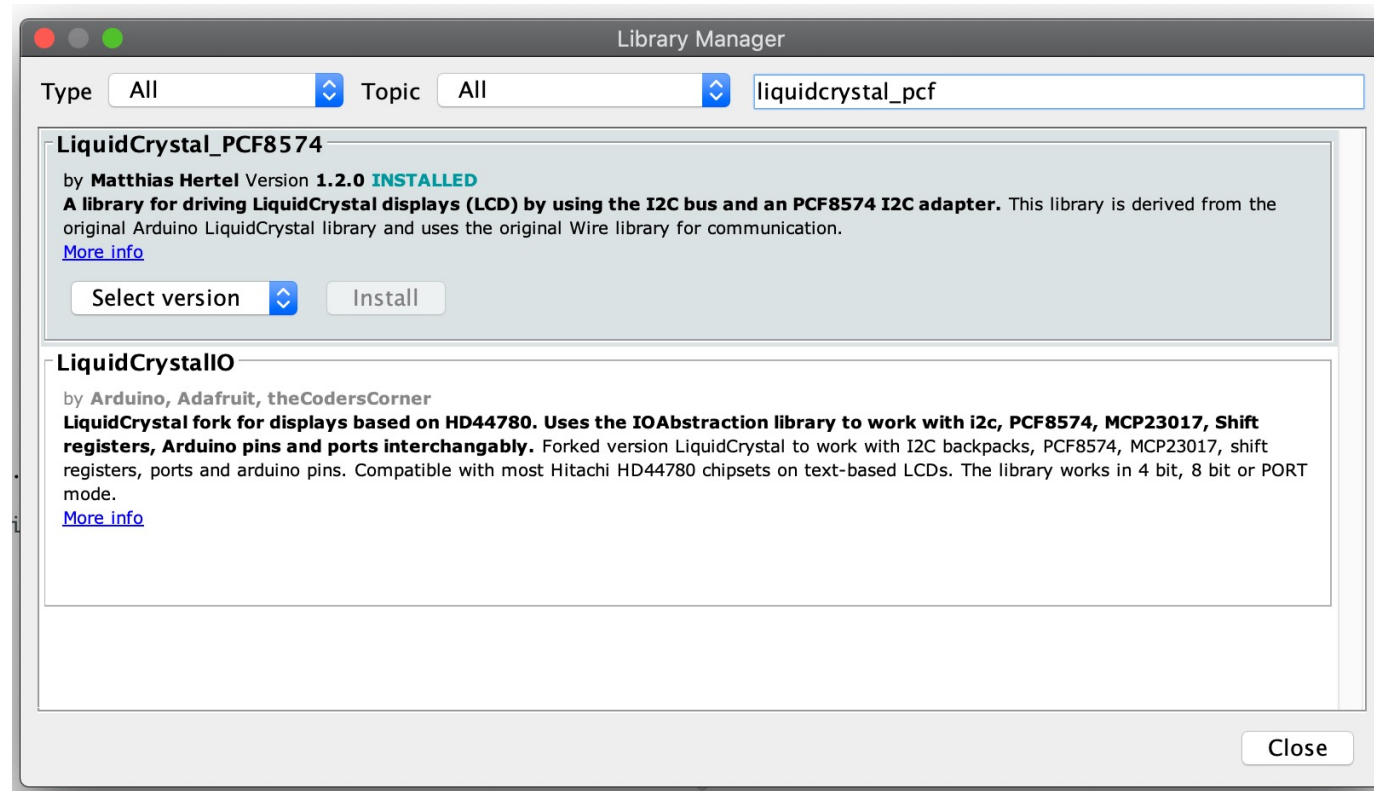
Careful: you have two "types" of LCDs, one has a different pin ordering (VCC, GND, SCL, SDA). **Check PIN names before connecting!!**



# LDC Library

In the Arduino IDE, select **Tools/Manage Libraries**.

Look for the `LiquidCrystal_PCF8574` library and install the latest version.





# LDC Library

Install the latest version of the `LiquidCrystal_PCF8574` library.

The library contains a lot of methods to control the LCD display via I2C

- `begin()` to initialise the display
- `clear()` clear the display content
- `home()` set the cursor to row 0, column 0
- `setCursor(row, col)` sets the cursor to row, col
- `setBacklight(brightness)` sets the brightness level (0 to 255)

You can inspect the source code directly to view the available methods.

# LDC Library

The display class defined in the library inherits from the `Print` class

- Which means that you can call the `print()` method to print strings directly
- You don't need to `write()` individual characters one by one.

The library is based on the `Wire` library for I2C communication.

- On the Arduino, this library uses pins A4/A5 by default

# New Code Element

Include the library and create a display class instance:

```
#include <LiquidCrystal_PCF8574.h>  
  
LiquidCrystal_PCF8574 lcd(0x27);  
  
-
```

0x27 is the I2C address of display.

# New Code Element

The datasheet says the address depends on the three jumper caps on the rear panel.

Some of you might have soldered pins instead.

You should all have either address **0x27** or **0x20** by default.

| A2 | A1 | A0 | IIC Address |
|----|----|----|-------------|
| 0  | 0  | 0  | 0x20        |
| 0  | 0  | 1  | 0x21        |
| 0  | 1  | 0  | 0x22        |
| 0  | 1  | 1  | 0x23        |
| 1  | 0  | 0  | 0x24        |
| 1  | 0  | 1  | 0x25        |
| 1  | 1  | 0  | 0x26        |
| 1  | 1  | 1  | 0x27        |

- 0: The Jumper Cap is connected
- 1: The Jumper Cap is disconnected

# New Code Element

In `setup()`, initialise the communication with the display, set the backlight and clear the screen. Then, print a message.

```
void setup() {  
    lcd.begin(16, 2);  
    lcd.setBacklight(255);  
    lcd.home();  
    lcd.clear();  
    lcd.print("Temperature:");  
}
```

# Exercise 6: Output Example





**Politecnico  
di Torino**

**Prof. Luca Barbierato**

Assistant Professor



**Electronic Design Automation Group**

Dipartimento di Automatica  
ed Informatica